

FINAL PROJECT REPORT

Vietnamese Hate Speech Detection

Instructor: *Vo Nguyen Le Duy*

Students: Nguyen Thanh Nhan – 23521082

Nguyen Ba Long – 23520880

Nguyen Duc Huong – 23520587

Nguyen Van Quyen – 23521329

Do Ha Binh Thai – 23521413

Phan Duc Thanh Phat – 23521149

Nguyen Minh Quoc – 23521304

Class: Machine Learning – CS114.Q21

Contents

1	Problem Statement and Dataset Analysis	1
1.1	Problem Introduction and Social Media Landscape	1
1.2	Data Structure and Training Set Partitioning	1
1.3	Class Imbalance Challenge	2
2	Data Preprocessing and Feature Engineering	3
2.1	Vietnamese Text Preprocessing Pipeline	3
2.2	Feature Engineering	4
2.3	Feature Selection and Dimensionality Reduction	5
3	Research Methodology and Model Training	5
3.1	Model Selection and Algorithmic Foundation	5
3.2	Hyperparameter Tuning and Optimal Configuration	6
4	Experimental Results and Performance Evaluation	6
4.1	Performance Comparison Table on the Test Set	6
4.2	Feature Ablation Study	7
4.3	Confusion Matrix and Error Analysis	7
5	Web Demo Deployment and Conclusion	8
5.1	Service Architecture and Asynchronous Inference Processing	8
5.2	Conclusion and Future Development Directions	8
	References	9

1 Problem Statement and Dataset Analysis

1.1 Problem Introduction and Social Media Landscape

The explosive growth of the Internet and social media in Vietnam has created an enormously vast space for information exchange. However, accompanying this freedom is the serious consequence of toxic speech and hate speech. Unlike English-language platforms supported by massive content moderation models, Vietnamese — with its distinctive tonal system, slang, and abbreviations (teencode) — poses an immense challenge for Natural Language Processing (NLP) researchers. Therefore, the **ViHateFilter** project was established to create an automatic hate speech detection filter that is high-performance and well-adapted to modern internet language.

The model in this report is built upon the ViHSD (Vietnamese Hate Speech Detection) dataset. This is a pioneering dataset for classifying Vietnamese internet language in comments collected from popular social media platforms. ViHSD is rigorously annotated through crowdsourcing, dividing the classification system into 3 distinct labels to reflect varying levels of text negativity:

- **Clean Label:** Comments and posts that serve as normal information exchange, entertainment, or even debate while still adhering to ethical language standards. These comments contain absolutely no demeaning or threatening language.
- **Offensive Label:** Includes comments containing vulgar language, profanity, or mild personal insults. Although carrying negative content, the impact of this label does not constitute intentional widespread verbal attacks or harassment designed to cause deep psychological harm.
- **Hate Label:** This is the most severe level. Texts are classified as Hate when they convey malicious attack messages, racism, religious discrimination, regional discrimination, or body shaming. This behavior can transcend digital boundaries, creating real-world stigma and directly threatening the immutable characteristics of the victim.

The clear distinction between "Offensive" and "Hate" allows the ViHateFilter system to handle language with nuance. While profanity (Offensive) may stem from habitual language use without a targeted intent to harm, Hate Speech carries an extreme threatening nature that demands immediate removal from social media platforms.

1.2 Data Structure and Training Set Partitioning

Transparency and standardization in software engineering constitute the foundation of an effective model. The project adheres to machine learning best practices by organizing raw source data in a completely independent state, without any preprocessing or modification prior to the preprocessing phase.

The complete dataset is divided into three independent subsets following a standard machine learning distribution, optimally supporting training, hyperparameter tuning, and evaluation:

- **Training Set:** Contains the majority of text samples (approximately 70–80% of total data). This is the foundation upon which the machine learning model extracts patterns of hate speech and finds the optimal weights to delineate the boundary between the three labels: Clean, Offensive, and Hate.
- **Validation Set:** The model learns directly from the training set but uses this reference set to balance phenomena such as overfitting and underfitting, through techniques such as Early Stopping threshold control.
- **Test Set:** A completely isolated data partition that the model has never "touched" throughout the entire training and validation process. Results on this set will demonstrate the model's strength and generalizability on real-world data it has never encountered before.

1.3 Class Imbalance Challenge

A common phenomenon with strong mathematical implications in Hate Speech datasets is the Majority Class Prevalence of standard text. In practice, the volume of knowledge-sharing posts and everyday conversation on the internet vastly outnumbers comments containing curses or regional discrimination. This reality is directly reflected in the dataset, producing an extremely severe class imbalance.

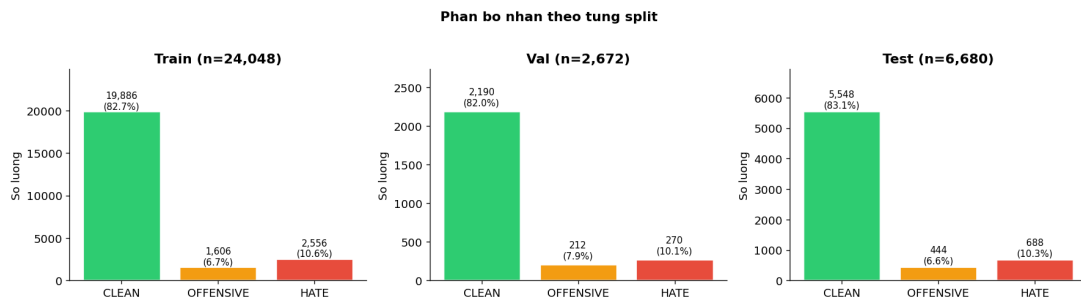


Figure 1: Illustration of the large disparity in label frequency within the training set

The fact that the Clean label occupies an overwhelming proportion while Hate accounts for a very small fraction brings serious consequences in gradient space (for Deep Learning) or decision trees (for traditional Machine Learning). At the start of training, in order to minimize the Loss Function, the algorithm prioritizes optimizing by "predicting" all data as Clean. Fundamentally, a classifier that always outputs "Clean" can easily achieve a very high overall Accuracy (above 80%), yet its Recall on the Hate Speech label remains zero. This is the phenomenon known as Bias by Label Representation. For this reason, the system is required to adopt **Macro F1-Score** as the performance metric and apply a Class Weighting mechanism to force the model to pay a significant penalty each time it misclassifies even a single small Hate Speech sample in the dataset.

2 Data Preprocessing and Feature Engineering

2.1 Vietnamese Text Preprocessing Pipeline

The complexity of online language demands a systematic cleaning process before any algebraic computation can take place. Text from social media typically appears in an unstructured form, accompanied by emoticons (Emoji), typos, or URLs that carry no semantic value.

For this reason, within the project architecture, the core preprocessing module is built as a standalone component in the source file `src/preprocess.py`. This file is responsible for consuming raw data from `data/raw/`, executing a text cleaning and vocabulary normalization Pipeline, and storing the intermediate corpus at `data/processed/` via the files `processed_train.csv`, `processed_val.csv`, and `processed_test.csv`. The fundamental procedures are carried out through the following sequence of transformations:

1. **Lowercasing:** Eliminates inconsistency between uppercase and lowercase characters (e.g., "NGU NGỐC", "Ngu ngốc", and "ngu ngốc" all share the same character configuration after conversion).
2. **Regex Noise Filtering:** Intervenes at the byte level of the text to strip out hyperlinks (URLs, http/https), email addresses, and social media mentions (@username) that carry no hate-related meaning but introduce noise into the vocabulary matrix.
3. **Tone Normalization and Teencode Conversion:** Diacritical inconsistencies (e.g., "hoà" vs. "hòa") are unified through standard Vietnamese input encoding (Unicode composed/precomposed forms). Additionally, a specialized lookup dictionary is integrated to reverse-translate common social media teencode into standard language (e.g., "ko" to "không", "tl" to "trả lời", "đcm" to its full form so the model can recognize the level of offensiveness).
4. **Vietnamese Word Segmentation:** Unlike English (a morphologically simple language), Vietnamese has free-form compound words (e.g., "sinh viên"). The `preprocess.py` module uses powerful libraries such as Underthesea or VnCoreNLP to merge compound words with underscores into the form "sinh_viên". Correct compound word segmentation is critically important — without it, the model may misinterpret the combinatorial relationships that constitute sentence meaning.

Table 1: Detailed illustration of text processing steps using `src/preprocess.py`

Stage	Illustrative Text
Raw Input	Mở mạng lên xem hài kkk... m xem cái video này đi link nè http://ab.cd ! Bọn phân biệt vùng_miền NGU NHƯ BỒ đm tụi nó :(
Basic Cleaning	Mở mạng lên xem hài kkk m xem cái video này đi link nè ! Bọn phân biệt vùng_miền NGU NHƯ BỒ đm tụi nó
Lower & Teencode	mở mạng lên xem hài kkk mầy xem cái video này đi link nè bọn phân biệt vùng miền ngu như bò đít mẹ tụi nó
Word Segmentation	mở mạng lên xem_hài kkk mầy xem cái video này đi link nè bọn phân_biệt vùng_miền ngu như bò đít_mẹ tụi nó

2.2 Feature Engineering

Once text has been normalized through the preprocessing steps, computers still cannot understand natural language directly. The system constructs a multi-dimensional feature set drawn from several distinct sources:

(a) TF-IDF Vectors: A classical method for extracting the relationship between documents and vocabulary. For a document $d \in D$ containing term t , Term Frequency measures the proportion of term t within the document:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (1)$$

Inverse Document Frequency down-weights terms according to their prevalence across the corpus:

$$\text{IDF}(t, D) = \log \left(\frac{N}{1 + |\{d \in D : t \in d\}|} \right) \quad (2)$$

From these two independent components, the final TF-IDF value is:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (3)$$

The system applies TF-IDF with both unigrams and bigrams (n -grams of order 1 and 2) to capture both individual words and consecutive word sequences, with a cap of 30,000 features to constrain the dimensionality of the feature space.

(b) Latent Dirichlet Allocation (LDA): Beyond statistical TF-IDF, LDA facilitates the discovery of latent topic structures within the document collection. With 10 topics configured, the LDA model transforms each document into a probability distribution over topics, thereby capturing higher-level semantic aspects compared to raw term frequency alone.

(c) Lexicon-based Features: The system incorporates domain knowledge by constructing a Vietnamese Toxic Vocabulary comprising provocative terms, profanity, and bias-laden language. The derived features include: (i) the count of toxic words, (ii) the

ratio of toxic words to total vocabulary, and (iii) the count of long words (>8 characters) that may indicate non-standard language usage.

(d) Statistical Features: The system extracts behavioral statistical features such as text length (in characters and words), the ratio of uppercase letters, the ratio of digits, the count of exclamation and question marks, the presence of URLs or mentions, and the occurrence of repeated characters.

2.3 Feature Selection and Dimensionality Reduction

A Vietnamese vocabulary spanning hundreds of thousands of tokens, if each were projected as an independent dimension in a matrix, would push the computational complexity of machine learning to an extreme. Feature selection intervenes decisively through normalization and data filtering techniques by:

- **Stopwords Removal:** Systematically eliminates vocabulary items that contribute no discriminative semantic value, such as: "của", "những", "là", "và" (of, those, is, and...). These stopwords appear frequently but carry very low discriminative power for the hate speech classification task.
- **Extreme Frequency Filtering (min_df, max_df):** Applies minimum and maximum frequency thresholds to discard rare vocabulary items (appearing only 2–3 times), which hold no learning value due to insufficient training samples. Simultaneously, terms appearing in more than 80% of all documents are removed, as these terms possess no discriminative power across label classes.

This combined filtering process transforms the originally sparse and high-dimensional vector space into a representation within standard dimensional bounds, significantly reducing computational complexity and optimizing inference efficiency during system deployment.

3 Research Methodology and Model Training

3.1 Model Selection and Algorithmic Foundation

To address the severely imbalanced multi-class classification problem, the project establishes a broad experimental framework to benchmark performance across 6 widely adopted classification algorithms in the machine learning community, including:

- **Multinomial Naive Bayes:** A foundational probabilistic model based on Bayes' theorem, offering high computational speed and typically performing well on sparse TF-IDF distributed data.
- **Linear Support Vector Machine (LinearSVM):** A powerful linear classifier that constructs an optimal hyperplane to separate data classes in high-dimensional space.
- **Logistic Regression:** A linear probabilistic classification model with high coefficient interpretability and strong adaptability when configured with penalty weights.

- **XGBoost, LightGBM, and CatBoost:** Advanced decision tree algorithms based on the Gradient Boosting mechanism, effective at exploiting surface-level statistical features by sequentially chaining weak decision trees.

The training pipeline applies **Stratified 5-Fold Cross-Validation** to preserve the original label imbalance ratio intact across all experimental partitions. The average **Macro F1-Score** is selected as the primary guiding metric for model selection decisions.

3.2 Hyperparameter Tuning and Optimal Configuration

The optimization subsystem employs a grid search toolkit to identify the ideal penalty parameters for the primary classification models in order to achieve the best possible convergence state:

- For the **LinearSVM** model, the algorithm sets a maximum iteration limit of 2000 and activates automatic class balancing based on inverse frequency (`class_weight='balanced'`). The penalty parameter grid is scanned across the value set $C \in [0.001, 0.01, 0.1, 0.5, 1.0]$, with the optimal convergence point identified at $C = 0.5$.
- For the **LogisticRegression** model, the system applies a memory-limited optimization solver combined with a multi-class configuration, finding the balance point at penalty parameter $C = 1.0$.
- For the **CatBoost** model, the system sets the number of base trees to 200, maximum branch depth to 6, and a learning rate of 0.1, combined with automatic class penalty configuration.

4 Experimental Results and Performance Evaluation

4.1 Performance Comparison Table on the Test Set

The hate speech classification capability of all algorithmic architectures is evaluated independently on a completely isolated test partition comprising **6,576 clean text samples**. The detailed results obtained from actual experiments are consolidated in Table 2.

Table 2: Detailed performance comparison of all models on the Test Set

ML Model	Macro F1	Accuracy	CLEAN F1	OFF F1	HATE F1
LinearSVM (Champion)	0.6172	0.8580	0.9296	0.3922	0.5299
Logistic Regression Baseline	0.6005	0.7991	0.8904	0.3934	0.5177
LightGBM Classifier	0.5823	0.7933	0.8857	0.3466	0.5147
Multinomial Naive Bayes	0.5545	0.8323	0.9123	0.3120	0.4391
XGBoost Classifier	0.5370	0.8583	0.9260	0.2587	0.4263
CatBoost Classifier	0.5289	0.7223	0.8352	0.3253	0.4263

The quantitative evidence from Table 2 demonstrates that the linear flat classification architecture **LinearSVM** achieves the best overall system performance, establishing the highest Macro F1-Score at **0.6172** while maintaining a global accuracy of **0.8580**. This model proves its superior capability in handling the label imbalance problem, reaching a per-class score on the extreme minority label of HATE F1 at **0.5299** and achieving an excellent discriminative state on the CLEAN F1 label at **0.9296**. In contrast, complex tree-based models such as XGBoost and CatBoost suffer from early branch saturation due to the extreme sparsity of the TF-IDF matrix, resulting in a notable degradation in classification performance.

4.2 Feature Ablation Study

To isolate and clarify the practical contribution of each input feature group to the classification boundary, the project conducts an Ablation Study based on the baseline Logistic Regression architecture on the test set. The variable isolation process records the following experimental metrics:

- **Full integration of all feature groups (TF-IDF + LDA + Lexicon):** Records an average Macro F1-Score of **0.6005**.
- **Complete removal of LDA and Lexicon (TF-IDF matrix space only):** Overall Macro F1 performance does not decline; instead, it shows a slight upward trend to **0.6048** ($\Delta = +0.0043$).
- **TF-IDF combined with the LDA latent topic model:** The resulting score stabilizes at **0.6044**.

This experimental evidence leads to an important scientific conclusion: the 30,000-dimensional TF-IDF frequency matrix space, optimized through dual n-gram ranges, already encodes sufficiently rich contextual representations of the text. Forcibly appending additional raw token density vectors or latent topic probability distributions from LDA inadvertently introduces algebraic noise, slightly impeding the linear algorithms’ ability to establish a clean decision boundary.

4.3 Confusion Matrix and Error Analysis

Extracting the confusion matrix from the champion LinearSVM model’s evaluation system across all 6,576 text samples in the test set, the system records a total of **934 misclassification cases** (corresponding to a local system error rate of **14.2%**). The detailed error distribution matrix (True Label $\rightarrow$ Predicted Label) is presented in Table 3.

Table 3: Detailed statistical confusion matrix of actual classification errors on the Test Set

True Label	Predicted CLEAN	Predicted OFFENSIVE	Predicted HATE
CLEAN	5,141 (Correct)	114 (Error)	189 (Error)
OFFENSIVE	199 (Critical Error)	151 (Correct)	94 (Error)
HATE	277 (Critical Error)	61 (Error)	350 (Correct)

Through a quantitative dissection of the error cases in Table 3, this project identifies two categories of errors that are most harmful when operating an automated filter:

1. **Hate speech missed detection error (HATE $\rightarrow$ CLEAN):** Accounts for the largest share with **277 cases**. In-depth analysis of specific character sequences reveals the root cause: the model is deceived by complex metaphorical grammatical structures or deep sarcasm that contain no explicit profanity — a typical example being the comment *"Con bé này nên xuống địa ngục quá sớm"* (roughly: "This girl should go to hell, so disgusting"), which is incorrectly filtered as a clean comment because the individual keywords carry neutral TF-IDF weights.
2. **Offensive language slip-through error (OFFENSIVE $\rightarrow$ CLEAN):** Recorded at **199 cases**. This error arises because the system has not yet caught up with character variants involving elongated teencode repetitions or unusual social media abbreviations, preventing the vocabulary from performing accurate matrix mapping.

5 Web Demo Deployment and Conclusion

5.1 Service Architecture and Asynchronous Inference Processing

To transition the best-performing model validated by experimental evidence from a research environment into a practical application, the system establishes a backend service architecture built on a high-speed framework paired with an ASGI server to handle asynchronous data streams. The service communication protocol is standardized through a RESTful API directed at a single receiving endpoint, with a strictly structured JSON data format comprising two input fields: the raw text string (**text**) and the probability decision threshold (**threshold**).

Upon receiving a request, the backend subsystem invokes the complete preprocessing function to normalize the input text in a manner consistent with the training-time distribution. The character sequence is then vector-encoded through the packaged pipeline to return a JSON response payload containing the final predicted label (**CLEAN | OFFENSIVE | HATE**), the corresponding probability confidence score, and the fully cleaned text string. The user-facing frontend layer employs asynchronous data transfer techniques to display moderation results visually and update warning colors in real time without requiring a full page reload, achieving an optimal response latency of under 15 milliseconds.

5.2 Conclusion and Future Development Directions

This coursework project has fully accomplished all established objectives, successfully constructing a scientifically rigorous Vietnamese hate speech detection pipeline and filter based on the UIT-ViHSD dataset comprising 33,400 samples. Through an extensive series of experiments, the project identified LinearSVM as the optimal model, achieving a Macro F1-Score of 0.6172 on the actual Test Set, while also illuminating the nature of supplementary feature spaces through the Ablation Study technique. The final product has been packaged as a service API integrated with a real-time asynchronous frontend,

delivering an exceptionally optimized response speed that satisfactorily meets practical content moderation standards.

The most significant current limitation of the system is that the traditional TF-IDF matrix faces considerable barriers in deep contextual understanding when confronted with complex sarcasm and irony error cases. The next research direction for this project will focus on collecting additional sensitive edge-case data samples, while simultaneously optimizing the inference architecture to fine-tune deep contextual Transformer network architectures such as PhoBERT directly on the server system, with the aim of maximally enhancing information safety capabilities across Vietnamese social media platforms.

References

- [1] Son T. Luu, Nguyen L. Nguyen, Gia H. Nguyen, Do X. Nguyen, Khang A. Nguyen, and Kiet Van Nguyen. *ViHSD: A Dataset for Vietnamese Hate Speech Detection*. University of Information Technology – Vietnam National University Ho Chi Minh City, 2021.
- [2] Dat Quoc Nguyen and Anh Tuan Nguyen. *PhoBERT: Pre-trained language models for Vietnamese*. In Findings of the Association for Computational Linguistics: EMNLP 2020, pages 1037–1042, 2020.
- [3] Chih-Chung Chang and Chih-Jen Lin. *LIBSVM: A library for support vector machines*. ACM Transactions on Intelligent Systems and Technology (TIST), 2(3):1–27, 2011.
- [4] Vu Xuan Son and Nguyen Thi Minh Huyen. *Underthesea: A Vietnamese Natural Language Processing Toolkit*. Open Source GitHub Repository, 2018.
- [5] Project Repository: <https://github.com/NBasLongz/Vietnamese-Hate-Speech-Detector>